LLM for Heuristics的应用

计算机科学与工程学院
School of computer science and engineering

Man-Made Heuristics Are Dead. Long Live Code Generators!

HotNets'25

Rohit Dwivedula, Divyanshu Saxena, Aditya Akella, Swarat Chaudhuri, Daehyeok Kim

The University of Texas at Austin

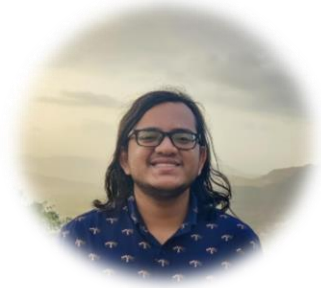
Austin, TX, USA

Reporter: Yuxiang Lu

UT Networked Systems

2026.01.28

作者团队



Rohit Dwivedula
Ph.D. student

Aditya Akella
Professor, Regents Chair, ACM Fellow

Daehyeok Kim
Assistant Professor

UT Networked Systems

UTNS研究组
研究大数据分析、分布式机器学习和大规模分布式应用的基础系统和网络问题。

Learning Directed Operating System (LDOS)
构建基于机器学习的操作系统，推动计算基础设施朝着高效率和高性能发展。

HotNets'25 - Man-Made Heuristics Are Dead. Long Live Code Generators!

IROS'25 - ConfigBot: Adaptive Resource Allocation for Robot Applications in Dynamic Environments

作者团队

Aditya Akella
Professor, Regents Chair, ACM Fellow

UT Networked Systems

LDOS

InfraAI

目前主要的研究方向是通过先进人工智能的视角重新思考系统基础设施

LDOS项目是NSF CISE Expedition in Computing项目，目前正在研究重新构想操作系统，以支持数据驱动的决策。

InfraAI中心是一个业界与 UT 合作的项目，旨在从零开始构建下一代人工智能的系统基础设施。

NSDI'26 - UNUM: A New Framework for Network Control
NSDI'26 - Synthesizing a Provably Performant Joint Controller for Low-Latency Video Streaming
NSDI'26 - SYMPHONY: Improving Memory Management for LLM Inference Workloads
NSDI'26 - Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC
NSDI'26 - Towards Performance Robustness for Microservices
NSDI'26 - Express Lane to Efficiency and Reliability: Multi-Dimensional Control in Meta' s Express Backbone Network
EuroSys'26 - Canopy: Property-Driven Learning for Congestion Control
HotNets'25 - Man-Made Heuristics Are Dead. Long Live Code Generators!
EMNLP'25 - Large Language Models as Realistic Microservice Trace Generators
IROS'25 - ConfigBot: Adaptive Resource Allocation for Robot Applications in Dynamic Environments
ACL'25 - StitchLLM: Serving LLMs, One Block at a Time
ICML'25 - HALoS: Hierarchical Asynchronous Local SGD over Slow Networks for Geo-Distributed Large Language Model Training
ICML'25 - On-the-Fly Adaptive Distillation of Transformer to Dual-State Linear Attention for Long-Context LLM Serving

目录

- 背景与动机
- 系统设计
- Case Study
- 总结与思考

目录

- 背景与动机
- 系统设计
- Case Study
- 总结与思考

背景与动机

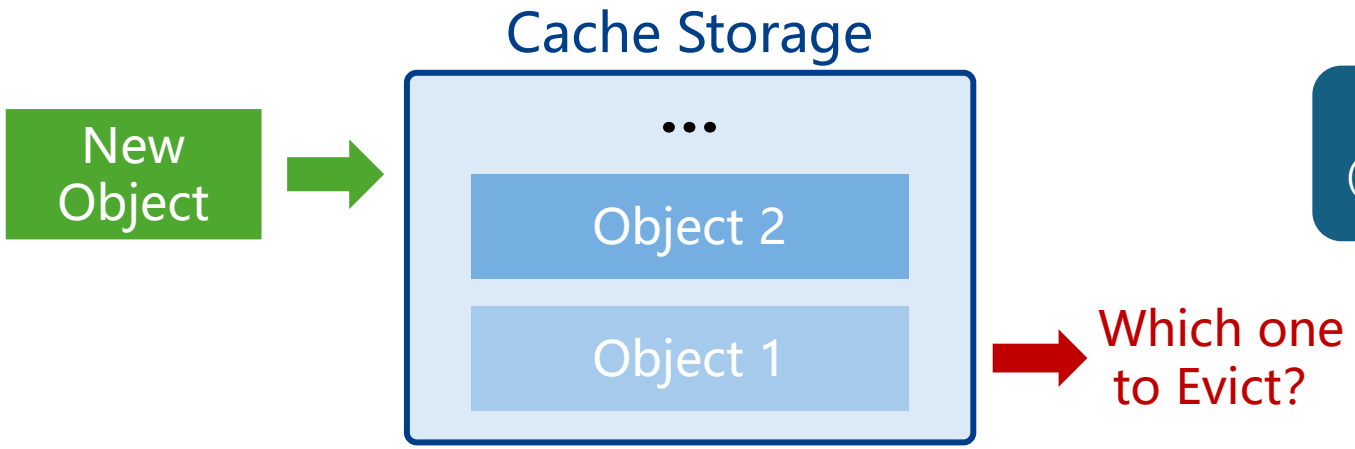
- **系统策略是计算机系统的决策核心**

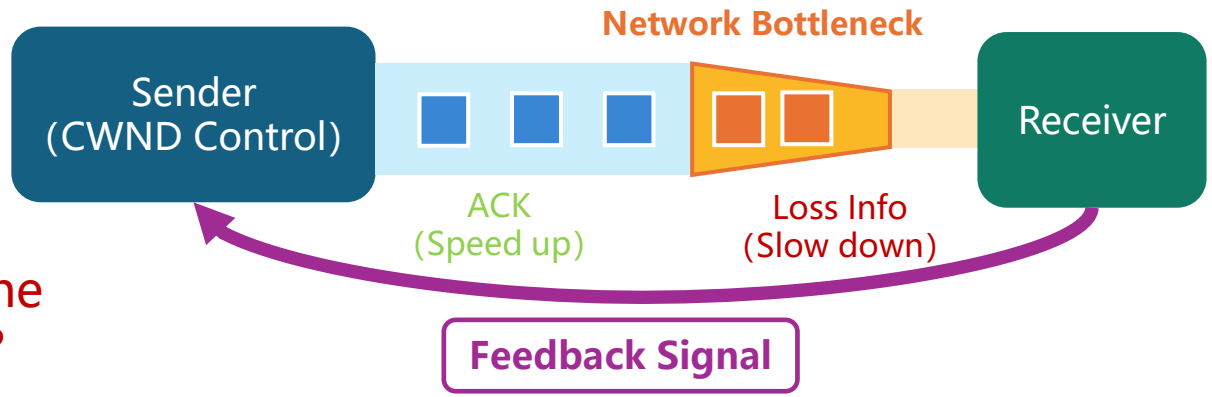
  系统性能往往取决于生成系统关键决策的算法的效果。

  关键场景：

  Web Caching (网页缓存)：决定“留谁删谁”，直接影响响应延迟与带宽成本。

  Congestion Control (拥塞控制)：决定“发包快慢”，在网络拥塞与吞吐量之间寻找平衡。





- **长期以来，策略设计依赖领域专家的经验与直觉**

  专家会针对特定场景（特定的工作负载或硬件）手工设计启发式算法。

背景与动机

- **这些高度定制的策略在新的工作负载、硬件或目标下会出现效果下降甚至完全不适用的问题**
- **传统的应对方案是不断调整和重构启发式方法**
- **“This is an arms race we are losing.”**

  随着工作负载、部署环境和硬件异构的快速演变，专家手工设计很难足够快地发现和设计出“最优”的策略了
- **因此，我们需要一种能够快速自动适应环境变化的方法！**

背景与动机

- **基于神经网络设计的策略已经证明，通过从数据中学习，能够实现接近实例最优的效果（即能够为特定情形找到最佳策略）。**
- **神经网络方法有很大的问题：**

  NN是黑盒模型，结果的可解释性很差

  复杂的训练和部署流程导致推理开销巨大

  安全问题

是否可以实现快速高效地设计“实例最优”的启发式算法

背景与动机

神经网络的权重生成 ✗ → 实例最优的代码生成 ✓

- 在代码生成的领域，最新的工作采用LLM引导的进化计算的方式实现生成
- 将策略设计问题转变为一个自动化搜索问题，不再由人来决定算法的结构，编写代码，而是直接由LLM搜索组合，生成算法的代码

摆脱固定的逻辑 ✓

探索广阔的策略空间 ✓

高效生成算法 ✓

未来愿景

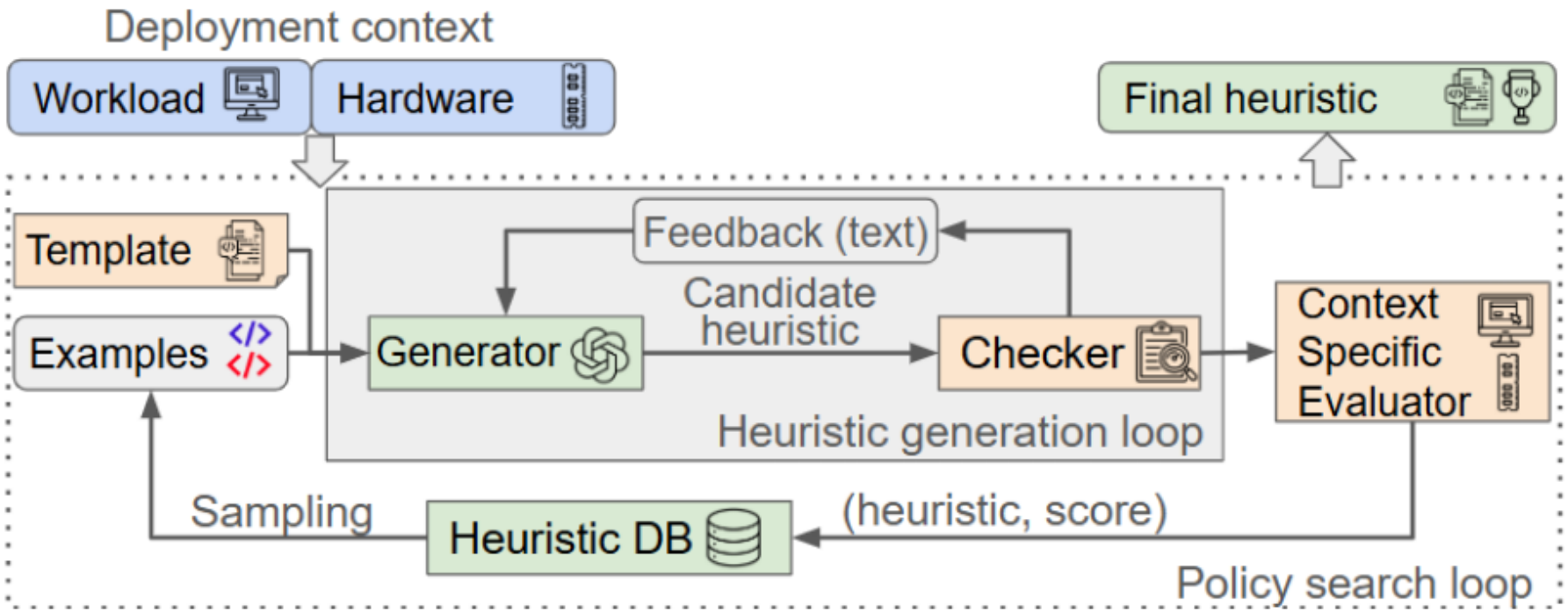
未来的系统将完全具备自我演进能力，只需要简单的上下文，就能够生成实例最优的启发式算法。此时开发者将不再需要亲自编写繁琐的启发式逻辑细节，而是可以将更多的精力放在定义问题的需求和设计高级的规范上面

目录

- 背景与动机
- **系统设计**
- Case Study
- 总结与思考

系统设计

- **PolicySmith系统架构**



两层循环结构

外层：策略搜索循环（Policy Search Loop）

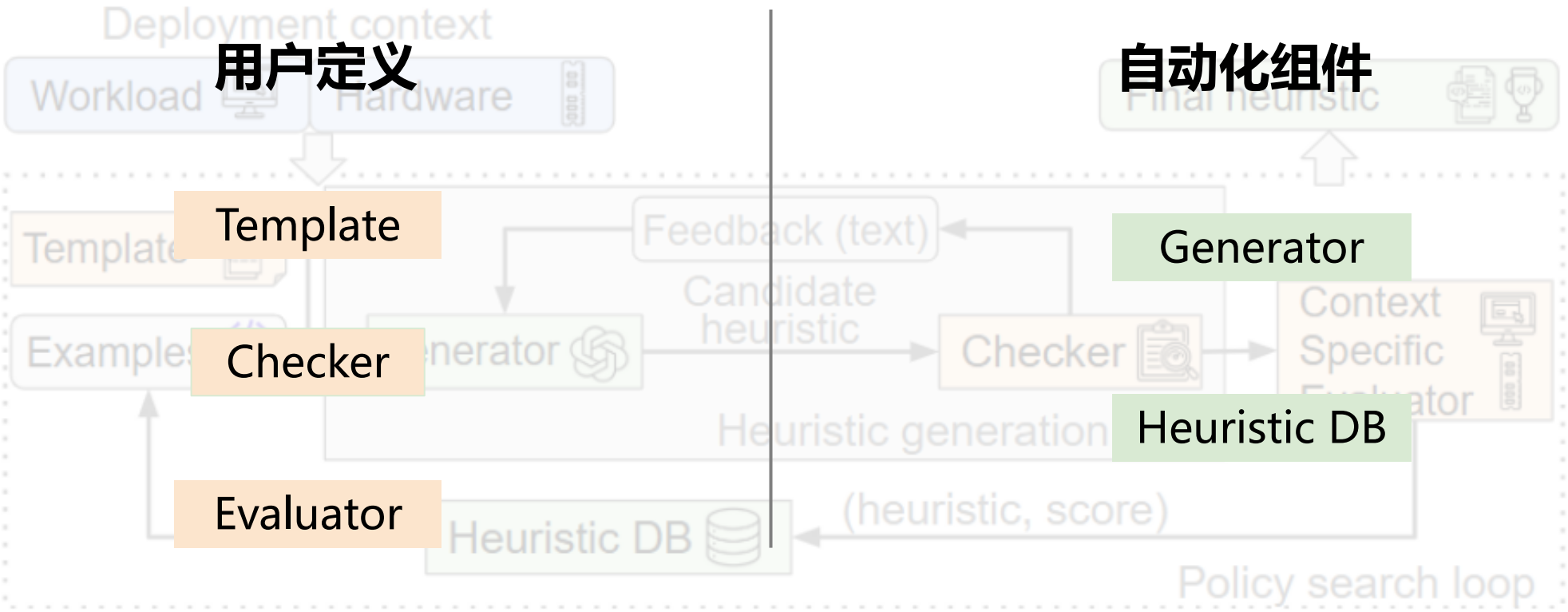
内层：算法的生成循环（Heuristic Generation Loop）

用户定义还是自动生成?

系统设计

- **PolicySmith系统架构**

用户定义还是自动生成?



用户定义

Template

Checker

Evaluator

自动化组件

Generator

Heuristic DB

目录

- 背景与动机
- 系统设计
- **Case Study**
- 总结与思考

Case 1：Web Caching

- **设计**

  Template

状态管理+决策逻辑

状态管理

- 形式：优先级队列
- 功能：追踪元数据
- 动作：在访问或插入时更新

固定

Per-object Aggregates History

决策逻辑

- 形式：优先级函数（priority()函数）
- 功能：计算队列中数据的优先级，选择剔出队列的数据

LLM生成

Case 1：Web Caching

- **设计**

Generator

采用轻量级大语言模型GPT-4o-mini作为生成核心

Checker

通过一个简单的检查器即可检查

Evaluator

在模板对应的真实trace上运行算法，计算未命中率

随着LLM的演进，未来的设计空间将会不断扩大，我们将可以针对越来越复杂的模板和越来越多的功能实现策略设计

Case 1：Web Caching

- **效果评估**

  设置

  数据集：两个来自真实世界的trace数据集：CloudPhysics 和Microsoft Research Cambridge（MSR）数据集

  Baseline：GDSF 、S3-FIFO 、SIEVE 、LIRS 、LHD 、Cacheus 、FIFO-Re、LeCaR 、SR-LFU 、CR-LRU 、LRU、MRU 和 FIFO，总共14种算法

```
priority(now, obj_id, obj_info, counts, ages,
    sizes, history):
  score = obj_info.count * 20;
  age   = now - obj_info.last_accessed;
  score -= age/300;
  score -= obj_info.size/500;
  if (history.contains(obj_id)){
    h = history.get_metadata(obj_id);
    score += h->count*15;
    score += h->age_at_eviction_time/150;
  }
  else score-=40;
  recent = ages.percentile(0.75);
  if (obj_info.last_accessed<recent) score-=30;
  big = sizes.percentile(0.75);
  if (obj_info.size > big) score-=25;
  else score+=10;
  frequent = counts.percentile(0.7);
  score += (obj_info.count>frequent) ? 50:-5;
  if (age < 1000) score+=25;
  if (obj_info.count < 3) score -= 15;
  return score;
```

Case 1：Web Caching

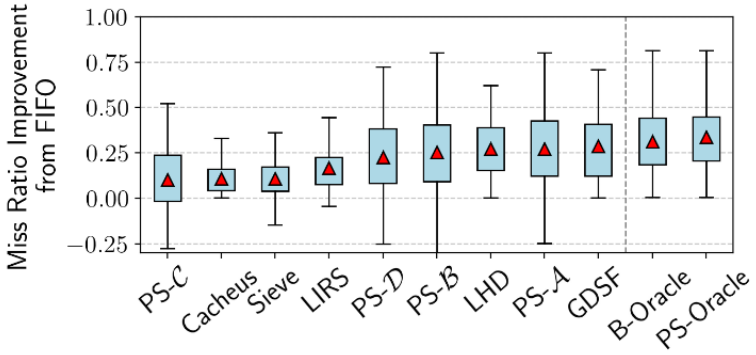
- **效果评估**

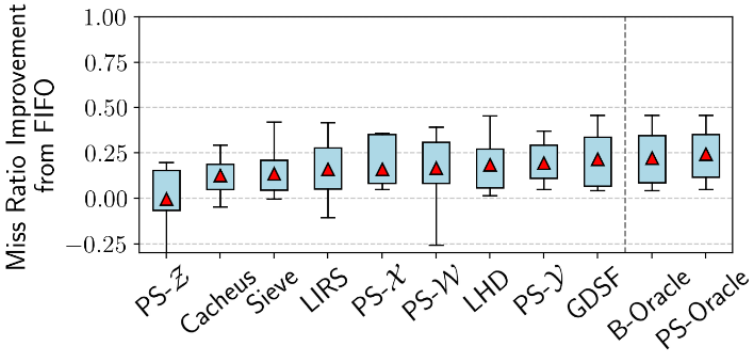
  表中的A-D，W-Z是 PolicySmith 生成的启发式方法。这些方法在上下文对应的trace上对比所有14个基线都表现出了要么相似要么更优的未命中率，这表明 PolicySmith 确实实现了针对每个上下文定制启发式算法。

  此外，作者还评估了每个 PolicySmith 生成的启发式算法，不仅针对其原始 trace ，还针对同一数据集内的所有其他trace进行评估。

  图中的结果展示了基线和 PolicySmith 生成的启发式算法对 CloudPhysics 和MSR数据集中所有 trace 的表现。

Dataset	%age of traces	
CloudPhysics	$\mathcal{A}$ (48%)	$\mathcal{B}$ (42%)
	$\mathcal{C}$ (14%)	$\mathcal{D}$ (31%)
MSR	$\mathcal{W}$ (57%)	$\mathcal{X}$ (64%)
	$\mathcal{Y}$ (57%)	$\mathcal{Z}$ (21%)





Case 2：Congestion Control

更具挑战性的场景

Linux 内核的系统设计包含各种人工调优的策略 (TCP拥塞控制, Packet Scheduling, I/O)，针对这些问题的启发式方法是数十年的特定领域研究的积累。

核心的挑战

① 内核编程环境受限 → 生成器的代码生成极其困难

② 内核开发有严格的安全和性能要求 → 检查器和评估器的设计变得至关重要

Case 2：Congestion Control

- **设计**

Template

固定

状态管理

- 当前的 TCP 状态，包括拥塞窗口 (cwnd)、最小往返时间 (min RTT)、飞行中字节数 (inflight bytes) 等
- 历史状态维护 (History Arrays)

LLM 生成

决策逻辑

- 生成 cong_control 函数，用于接收状态并计算新的cwnd大小
- 为了安全这一生成会被卸载到一个动态挂在的eBPF探针

Checker

每当函数运行时，探针就会被自动触发，计算更新后的结果，写入eBPF映射

Case 2：Congestion Control

- **初步结果**

生成了100个候选拥塞控制启发式方法，并尝试将它们汇编成eBPF程序。只有63%的候选方法首次尝试就通过eBPF验证器，另外有19%只是成功编译。

内核代码的编译率远低于缓存任务的编译率：Web caching任务中92%的候选代码在第一遍就编译完成。

作者在12 Mbps、20毫秒延迟模拟链路上评估了成功编译的启发式方法的效果。结果行为差异很大：带宽利用率从23%到98%不等，平均排队延迟从2毫秒到40毫秒不等。**这种差异说明了可以通过PolicySmith这样的自动化方法探索多样化的拥塞控制策略。**

目录

- 背景与动机
- 系统设计
- Case Study
- **总结与思考**

总结与思考

- **总结**

本文针对系统策略设计领域，需求快速发展，专家手工设计无法高效生成有效的启发式算法的问题，提出了 PolicySmith 框架。该框架将策略设计重定义为自动化搜索问题，通过结合 LLM 引导的进化搜索，依照专家精心设计的模板和上下文提示，PolicySmith 的原型系统成功在 Web 缓存和拥塞控制任务上实现了根据上下文高效生成实例最优的启发式算法。

总结与思考

- **基于本文，还能做什么?**

基于 PolicySmith 的现有成果，我认为未来有两个方向可以进行探索：

1. 从“离线训练”转向“在线自适应”，文章中也说明了，目前的 PolicySmith 原型系统只是聚焦于根据上下文生成对应的算法这一点，但是实际应用过程中为了适应实时变化的工作负载，会需要配合专门的变化感知“触发器”，如何形成这样的统一系统是一个可以探索的方向。
2. 从“单点优化”转向“联合优化”，目前 PolicySmith 的实现仅局限于单一组件的优化，而系统设计中的组件之间往往存在联系，从整体上设计相互关联的组件策略十分必要，鉴于 LLM 的能力，可能可以采用多智能体协作的框架。

总结与思考

• 方法的泛化性

当我们去思考算法设计的泛化性的时候，我们往往会陷入一个误区，我们要设计一个通用的算法来应对所有的问题，这往往会损失效果。但 PolicySmith 带来了一个关键的思想转折：**真正的泛化性不应体现在最终的“策略代码”上，而应体现在“生成过程”中。**

通过引入 LLM，我们能够实现将“通用性”上移了一层：我们不再追求一套代码逻辑在不同场景间的迁移，而是追求一个“context”设计一套代码逻辑，这种思想使得我们能够对每一个细分的、长尾的特定场景，都能低成本地定制出专属的 SOTA 模型，彻底解决了通用算法在特定场景下效果下降的问题。

总结与思考

• 对自己工作的启发

PolicySmith 的实践对我的启发最大的一点其实是在如何通过科学地解构问题来确定**“人机协作边界”的精准架构**，将不可妥协的系统约束（LLM不可生成的部分）与广阔的探索空间（LLM可以自由生成的部分）合理剥离。单纯的代码编写和模型设计已经不再是最大的瓶颈。因为我自己的工作想要做的是在任务调度领域的LLM for RL，目前在进行的其实也是对模板和搜索空间的设计。

谢谢老师同学们的聆听!

欢迎提问交流!